

实验细节

本次实验包含 2 个任务，包括重写 `sys_get_time` 和 `sys_trace` 系统调用，实现 `mmap` 和 `munmap` 的匿名映射功能。

由于加入了虚拟内存的机制，所以在处理 `sys_get_time` 和 `sys_trace` 时不能直接使用内核的物理地址，而是需要使用进程的虚拟地址所对应的物理地址。我添加了一个 `copy_to_user` 函数来实现这个功能。

`mmap` 和 `munmap` 两个系统调用需要操作进程的 `MemorySet` 结构体，以此在操作系统中分配和注册内存区域，否则无法使用 `FrameTracker` 对物理内存的分配和释放进行管理。因此我实现 `map_mem_area` 和 `unmap_mem_area` 函数来实现这两个系统调用。

问答题

问题一

答：如下表所示，SV39 的页表项共 64 位，根据《The RISC-V Reader》提供的信息，各项分别占用如下：

Reserved	PPN[2]	PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V
10	26	9	9	2	1	1	1	1	1	1	1	1

Table 1: RV39 页表项

1. Reserved: 保留位，保留给未来使用
2. PPN[2]: 一级页表中，对二级页表的页表项的索引
3. PPN[1]: 二级页表中，对三级页表的页表项的索引
4. PPN[0]: 二级页表中，对叶子页表项的索引
5. RSW: 保留给操作系统使用,硬件将忽略该字段
6. D: 脏位，表示该页表项是否被修改
7. A: 访问位，表示该页表项是否被访问
8. G: 全局位，表示该页表项是否是全局页表项
9. U: 用户位，表示该页表项是否是用户页表项
10. X: 执行位，表示该页表项是否可执行
11. W: 写入位，表示该页表项是否可写
12. R: 读取位，表示该页表项是否可读
13. V: 有效位，表示该页表项是否有效

不过在我现在写的操作系统中，还不存在读写状态位的概念，还有 PPN[2]只考虑了 9 位。

问题二

1. 请问哪些异常可能是缺页导致的？

答：不太能够理解什么叫“哪些异常可能是缺页导致的”。但是我参考了《Linux 内核深度解析》中 3.14 节得知，以下情况可能导致缺页异常：

- 访问用户栈时，超出了栈表示的范围大小。
- 内核的 Lazy 策略使得，第一次访问时没有分配物理页。
- 内存不足时，内核把进程匿名页换出到交换区。
- 访问了一个没有映射的虚拟地址。

2. 发生缺页时，描述相关重要寄存器的值。

答：`scause` 以及用于内核态切换的 `stval`，`satp`，`sscratch`，`sepc` 寄存器的值将会变化。

- `scause` 寄存器：参考 [The RISC-V Instruction Set Manual: Volume II: Privileged Architecture](#)，当发生缺页异常时，`scause` 的值如下：
 1. **12**：指令获取时发生缺页
 2. **13**：加载操作时发生缺页
 3. **15**：存储操作时发生缺页
- `stval`，`satp`，`sscratch`，`sepc` 是用于从用户态切换至内核态，参考上一章。

3. 这样做有哪些好处？

答：使用 lazy 策略，可以有效地利用物理内存，不浪费空间给用不到的内存区域。

4. 处理 10G 连续的内存页面，对应的 SV39 页表大致占用多少内存 (估算数量级即可)？

答：10GB $\approx 2^{34}$ B，假设页大小为 4KB，则需要 $\frac{2^{34}}{2^{12}} = 2^{22}$ 个页表项。每个页表项占用 8 字节，所以大约需要 $2^{22} * 8 = 2^{25} B = 32MB$ 的内存。1，2，3 级页表的大小差异 10 倍以上，可以忽略不计。

5. 请简单思考如何才能实现 Lazy 策略，缺页时又如何处理？描述合理即可，不需要考虑实现。

答：在页表项中添加一位 **P** (Physics)位，表示当前页是否被分配物理页。当用户申请时，正常分配虚拟页，但是不分配物理页，此时 **P** 位为 0。当用户访问该页时，检查 **P** 位，如果为 0，则分配物理页，并将 **P** 位置为 1。这样就实现了 Lazy 策略。

6. 此时页面失效如何表现在页表项(PTE)上？

答：可以使用冗余的 **RSW** 位进行表示。

问题三

1. 在单页表情况下，如何更换页表？

答：修改 `satp` 寄存器的值，使其指向新的根页表的物理地址。

2. 单页表情况下，如何控制用户态无法访问内核页面？(tips:看看上一题最后一问)

答：可以使用 **U** 位来控制用户态是否可以访问内核页面。

3. 单页表有何优势？(回答合理即可)

答：相比 KPTI，单页表机制实现简单，不需要维护两套页表，可以节省一些性能开销。

4. 双页表实现下，何时需要更换页表？假设你写一个单页表操作系统，你会选择何时更换页表（回答合理即可）？

答：更换页表时，必然是因为用户程序需要操作系统的资源或者是遇到了中断。因此系统调用，中断处理和异常处理时，需要从用户态切换到内核态，此时需要更换页表。假设我写一个单页表操作系统，我会选择在系统调用时更换页表，用来处理必要的请求。

荣誉准则

1. 在完成本次实验的过程（含此前学习的过程）中，我曾分别与 以下各位 就（与本次实验相关的）以下方面做过交流，还在代码中对应的位置以注释形式记录了具体的交流对象及内容：
 - 暂无
2. 此外，我也参考了 以下资料，还在代码中对应的位置以注释形式记录了具体的参考来源及内容：
 - 《RISC-V 开放架构设计之道 1.0.0 (原著 The RISC-V Reader: An Open Architecture Atlas)》
 - 《RISC-V Supervisor Binary Interface Specification》
 - KiMi, Deepseek
 - 《Linux 内核深度解析》
 - 《The RISC-V Instruction Set Manual: Volume II》
3. 我独立完成了本次实验除以上方面之外的所有工作，包括代码与文档。我清楚地知道，从以上方面获得的信息在一定程度上降低了实验难度，可能会影响起评分。
4. 我从未使用过他人的代码，不管是原封不动地复制，还是经过了某些等价转换。我未曾也不会向他人（含此后各届同学）复制或公开我的实验代码，我有义务妥善保管好它们。我提交至本实验的评测系统的代码，均无意于破坏或妨碍任何计算机系统的正常运转。我清楚地知道，以上情况均为本课程纪律所禁止，若违反，对应的实验成绩将按“-100”分计。